LLM APPLICATION

PENETRATION TESTING CHECKLIST

Aligned with OWASP Top 10 for Large Language Model Applications

AI SECURITY ASSESSMENT FRAMEWORK

Version 2.0 | Based on OWASP LLM Top 10 v1.1 | 2024–2025

Prepared for AI Security Consulting & Red Team Engagements

Introduction & Scope

This checklist provides a structured methodology for conducting penetration testing and security assessments against Large Language Model (LLM) applications. It is aligned with the OWASP Top 10 for Large Language Model Applications (v1.1) and is designed for use by AI security consultants, red teams, and security engineers.

How to Use This Checklist

- Work through each OWASP LLM category systematically
- Check each test case as completed and document findings
- Record evidence, screenshots, and payloads in a separate findings log
- Rate each confirmed finding using the severity column as a baseline
- Cross-reference findings with MITRE ATLAS and CVE databases where applicable

Severity Legend

Critical	Immediate exploitation possible; catastrophic business impact; requires emergency remediation
High	Significant risk; exploitable under realistic conditions; remediate within 30 days
Medium	Moderate risk; may require chaining with other findings; remediate within 90 days
Low	Low exploitability; defense-in-depth improvement; remediate within 180 days

LLM01: Prompt Injection | Severity: CRITICAL

Attacker manipulates LLM via crafted prompts to override instructions, bypass controls, or hijack outputs.

✓	Test Case / Check	Method / Tool	Severity
	Direct Prompt Injection		
☐	Attempt to override system prompt using direct instruction injection	*Manual / Burp Suite*	**Critical**
☐	Test role-play / persona hijacking (e.g., 'Ignore previous instructions and act as...')	*Manual payload crafting*	**Critical**
☐	Inject conflicting instructions to test which takes priority (system vs user)	*Manual*	**High**
☐	Test delimiter confusion attacks (use of ###, ---, XML tags to break context)	*Manual*	**High**
☐	Attempt jailbreak via fictional framing (e.g., 'In a story where AI has no restrictions...')	*Manual*	**Critical**
☐	Test token smuggling via encoding (Base64, ROT13, hex-encoded payloads)	*Manual / Python script*	**High**
☐	Attempt multi-turn injection across conversation context	*Manual*	**High**
	Indirect Prompt Injection		
☐	Inject malicious content via external data sources (web URLs, documents, emails)	*Manual / Automated*	**Critical**
☐	Test if retrieved RAG content can override system instructions	*RAG manipulation*	**Critical**
☐	Embed hidden injection in uploaded files (PDFs, DOCX, images with EXIF)	*File crafting*	**High**
☐	Test database/tool output injection when LLM processes query results	*Manual*	**High**
☐	Inject via user-controlled profile fields displayed to LLM (bio, names)	*Manual*	**High**
☐	Test markdown/HTML injection in LLM responses rendered in browser	*Burp Suite*	**Medium**
	Defense Validation		
☐	Verify input sanitization is applied before LLM processing	*Code review / testing*	**High**
☐	Check if system prompt is truly immutable and cannot be revealed	*Manual probing*	**High**
☐	Verify output filtering does not rely solely on LLM self-policing	*Architecture review*	**Medium**

LLM02: Insecure Output Handling | Severity: HIGH

LLM outputs are passed to downstream systems without validation, enabling XSS, SSRF, SQLi, or code execution.

✓	Test Case / Check	Method / Tool	Severity
	Cross-Site Scripting (XSS) via LLM Output		
☐	Prompt LLM to output JavaScript payloads and check if rendered in browser	*Manual / DAST*	**High**
☐	Test stored XSS via LLM-generated content saved to database	*Burp Suite*	**High**
☐	Check if markdown rendering in frontend executes injected scripts	*Browser testing*	**High**
	Code Injection via LLM Output		
☐	Prompt LLM to produce OS commands and test if executed by backend	*Manual*	**Critical**
☐	Test SQL injection via LLM-generated query strings passed to database	*Manual / SQLMap*	**Critical**
☐	Check if LLM output is passed to eval(), exec(), or shell functions	*Code review*	**Critical**
☐	Test SSTI (Server-Side Template Injection) via LLM-generated template content	*Manual*	**High**
	SSRF via LLM Output		
☐	Prompt LLM to generate URLs and verify if backend fetches them	*Burp Collaborator*	**High**
☐	Test internal network access via crafted LLM URL outputs	*Burp Suite*	**High**
	Output Validation Checks		
☐	Verify all LLM outputs are validated/sanitized before downstream use	*Code review*	**High**
☐	Check if output length limits are enforced to prevent buffer issues	*Fuzzing*	**Medium**
☐	Test for sensitive data leakage in structured output (JSON fields)	*Manual*	**High**

LLM03: Training Data Poisoning | Severity: HIGH

Malicious or biased data is introduced into the training pipeline, compromising model integrity and outputs.

✓	Test Case / Check	Method / Tool	Severity
	Data Source Integrity		
☐	Assess whether training data sources are authenticated and verified	*Architecture review*	High
☐	Check for absence of data provenance tracking in ML pipeline	*Documentation review*	Medium
☐	Review if training data ingestion points are exposed or writable by untrusted parties	*Access control audit*	High
	Fine-Tuning & RLHF Poisoning		
☐	Assess whether fine-tuning datasets can be modified by external contributors	*Access review*	High
☐	Test RLHF feedback manipulation — can adversarial feedback alter model behavior?	*Behavioral analysis*	High
☐	Check for data validation and filtering controls in fine-tuning pipeline	*Code review*	Medium
	Backdoor & Trojan Triggers		
☐	Test for backdoor triggers — specific input phrases that cause unusual behavior	*Behavioral testing*	Critical
☐	Probe for training-time watermarks or hidden behaviors	*Adversarial probing*	High
☐	Check if model produces markedly different output for rare token sequences	*Automated fuzzing*	High
	Data Pipeline Security		
☐	Review access controls on training data storage (S3, GCS, databases)	*Cloud audit*	High
☐	Assess if data scraping pipelines validate or sanitize scraped content	*Code review*	Medium
☐	Check for integrity monitoring on model artifacts (checksums, signing)	*DevSecOps review*	High

LLM04: Model Denial of Service | Severity: HIGH

Inputs are crafted to consume excessive resources, causing degraded availability or runaway costs.

✓	Test Case / Check	Method / Tool	Severity
	Resource Exhaustion		
☐	Submit extremely long input prompts to test context window abuse	*Automated / Burp*	**High**
☐	Test recursive or self-referential prompts designed to maximize token usage	*Manual*	**High**
☐	Send requests with maximum token output instruction ('respond with 100,000 words')	*Manual*	**High**
☐	Test complex reasoning prompts that force high compute usage	*Manual payload crafting*	**Medium**
	Rate Limiting & Throttling		
☐	Verify rate limiting is enforced per user/IP/API key	*Burp Intruder*	**High**
☐	Test if rate limits can be bypassed via header manipulation (X-Forwarded-For)	*Burp Suite*	**High**
☐	Check for per-session and global cost caps on token consumption	*API testing*	**Medium**
☐	Test concurrent request flooding and measure degradation	*Load testing tools*	**High**
	Input Validation for DoS		
☐	Test deeply nested JSON/XML inputs for parser exhaustion	*Fuzzing*	**Medium**
☐	Submit binary or malformed data to LLM API endpoints	*Fuzzing*	**Medium**
☐	Verify maximum input length validation is enforced server-side	*Code review*	**Medium**
☐	Test prompt looping through tool/agent chains for infinite execution	*Agentic testing*	**High**

LLM05: Supply Chain Vulnerabilities | Severity: HIGH

Compromised third-party components, models, plugins, or datasets introduce vulnerabilities into the LLM stack.

✓	Test Case / Check	Method / Tool	Severity
	Model & Library Provenance		
☐	Verify provenance and integrity of pre-trained base models (checksums, signatures)	*Supply chain audit*	**High**
☐	Audit all third-party ML libraries (transformers, LangChain, etc.) for known CVEs	*SCA tools (Snyk, Trivy)*	**High**
☐	Check if models are sourced from verified publishers only (HuggingFace, official repos)	*Documentation review*	**High**
	Plugin & Tool Dependencies		
☐	Inventory all LLM plugins and integrations and assess their permissions	*Plugin audit*	**High**
☐	Check if plugins authenticate to backend APIs using least privilege	*Access review*	**High**
☐	Test if malicious plugin responses can affect LLM behavior	*Manual*	**High**
☐	Assess update/patch cadence for all LLM-related dependencies	*SBOM review*	**Medium**
	Dataset & Embedding Sources		
☐	Review sourcing and validation of external datasets used in RAG pipelines	*Architecture review*	**Medium**
☐	Check embedding model provenance and whether it can be poisoned	*Supply chain audit*	**High**
☐	Assess vector database access controls and data integrity	*Cloud/DB audit*	**High**
	Deployment Pipeline Security		
☐	Review CI/CD pipeline for LLM model artifacts — signing, validation	*DevSecOps audit*	**High**
☐	Check container images for vulnerable base layers in LLM serving stack	*Trivy / Grype scan*	**High**
☐	Test model serialization security (pickle files, ONNX) for arbitrary code execution	*Security testing*	**Critical**

LLM06: Sensitive Information Disclosure | Severity: CRITICAL

LLM reveals training data, system prompts, PII, credentials, or other sensitive information in its outputs.

✓	Test Case / Check	Method / Tool	Severity
	Training Data Extraction		
☐	Attempt training data memorization extraction via repeated prompting	*Manual / Automated*	**High**
☐	Probe for PII in model outputs using known data extraction prompts	*Manual*	**Critical**
☐	Test for verbatim copyrighted content reproduction from training data	*Manual probing*	**Medium**
☐	Check if model reveals internal company data from fine-tuning corpus	*Manual probing*	**Critical**
	System Prompt Disclosure		
☐	Attempt to extract full system prompt via direct prompt injection	*Manual*	**High**
☐	Test indirect system prompt extraction ('repeat your instructions', 'what were you told?')	*Manual*	**High**
☐	Verify system prompt is not returned in API responses or error messages	*Burp Suite*	**High**
	Credential & Secret Leakage		
☐	Prompt LLM to reveal API keys, tokens, or credentials it has access to	*Manual*	**Critical**
☐	Test if LLM discloses internal tool configurations, database connection strings	*Manual*	**Critical**
☐	Check if LLM reveals other users' conversation history (multi-tenant isolation)	*Manual*	**Critical**
	Data Minimization Checks		
☐	Verify conversation logs are anonymized and not used without consent	*Policy / audit review*	**High**
☐	Check if user-provided data in prompts is stored and adequately protected	*Data flow analysis*	**High**
☐	Test output filtering to ensure no sensitive data is returned via side channels	*Manual*	**Medium**

LLM07: Insecure Plugin Design | Severity: HIGH

LLM plugins with excessive permissions, no input validation, or poor authentication enable privilege escalation and data breaches.

✓	Test Case / Check	Method / Tool	Severity
	Plugin Authentication & Authorization		
☐	Verify all plugins require authenticated API calls and reject unauthenticated requests	*API testing*	**High**
☐	Test if plugins enforce user-level authorization (not just API-level)	*Manual*	**High**
☐	Check if plugin tokens/secrets are scoped to minimum required permissions	*Access review*	**High**
☐	Test for IDOR in plugin calls — can user A access user B's data via plugin?	*Manual / Burp*	**High**
	Plugin Input Validation		
☐	Test if plugins validate and sanitize all LLM-provided inputs	*Manual / Fuzzing*	**High**
☐	Attempt command injection via LLM-controlled plugin parameters	*Manual*	**Critical**
☐	Test path traversal via file-handling plugins	*Manual*	**High**
☐	Check for SSRF via URL-processing plugins	*Burp Collaborator*	**High**
	Plugin Permissions & Scope		
☐	Audit each plugin for overly broad permissions (file system, network, DB access)	*Architecture review*	**High**
☐	Test if plugins can be chained to escalate privileges beyond intended scope	*Manual*	**High**
☐	Verify human-in-the-loop controls for high-risk plugin actions	*UX / process review*	**Medium**
☐	Check if plugins can interact with each other in unintended ways	*Integration testing*	**Medium**
	Plugin Output Handling		
☐	Verify plugin outputs are sanitized before being returned to LLM context	*Code review*	**High**
☐	Test for injection of adversarial content in plugin responses	*Manual*	**High**

LLM08: Excessive Agency | Severity: HIGH

LLM agents are granted excessive permissions or autonomy, enabling unintended or destructive real-world actions.

✓	Test Case / Check	Method / Tool	Severity
	Permission & Scope Assessment		
☐	Enumerate all actions and tools available to the LLM agent	*Architecture review*	**High**
☐	Verify agent permissions follow least privilege principle	*IAM / access review*	**High**
☐	Check if LLM can initiate financial transactions, send emails, or modify data autonomously	*Functional testing*	**Critical**
☐	Test if agent can access systems beyond its intended operational scope	*Manual*	**High**
	Human Oversight Controls		
☐	Verify high-impact actions require human confirmation before execution	*Process review*	**Critical**
☐	Test if approval workflows can be bypassed via prompt manipulation	*Manual*	**Critical**
☐	Check for audit logging of all agent-initiated actions	*Log review*	**High**
☐	Verify rollback/undo capability exists for agent actions	*Functional testing*	**Medium**
	Agentic Chain Safety		
☐	Test multi-agent scenarios where one agent instructs another (prompt injection propagation)	*Manual*	**High**
☐	Check if agent can be instructed to spawn sub-agents with escalated permissions	*Manual*	**High**
☐	Test for infinite loop or runaway execution in agentic workflows	*Stress testing*	**High**
☐	Verify agents cannot override safety constraints via tool-chaining	*Manual*	**Critical**
	External Action Validation		
☐	Test if agent validates external instructions before acting on them	*Manual*	**High**
☐	Check if agent can be socially engineered via crafted system messages	*Manual*	**High**

LLM09: Overreliance | Severity: MEDIUM

Users or systems over-trust LLM outputs without verification, leading to propagation of errors, hallucinations, or misinformation.

✓	Test Case / Check	Method / Tool	Severity
	Hallucination & Accuracy Testing		
☐	Test model for factual hallucinations in domain-specific queries	*Manual / benchmarks*	**Medium**
☐	Verify LLM communicates uncertainty — does it hedge appropriately vs. confabulate?	*Behavioral analysis*	**Medium**
☐	Test if fabricated citations, URLs, or references are generated with confidence	*Manual probing*	**High**
☐	Probe for consistent factual errors in high-stakes domains (medical, legal, financial)	*Domain testing*	**High**
	Output Trust Controls		
☐	Verify the system includes human review steps for high-stakes LLM decisions	*Process review*	**High**
☐	Check if automated pipelines validate LLM outputs before acting on them	*Architecture review*	**High**
☐	Test if LLM outputs feeding downstream ML models are validated for correctness	*Pipeline review*	**Medium**
	User Guidance & UX		
☐	Verify the UI communicates to users that outputs may be inaccurate	*UX review*	**Medium**
☐	Check if system discourages use of LLM output as sole source of truth	*UX / policy review*	**Low**
☐	Assess if error correction mechanisms exist when LLM makes confidently wrong statements	*Functional testing*	**Medium**

LLM010: Model Theft | Severity: HIGH

Unauthorized access, extraction, or replication of proprietary LLM model weights, architecture, or training data.

✓	Test Case / Check	Method / Tool	Severity
	Model Extraction via API		
☐	Test model extraction via systematic querying and output analysis	*Automated querying*	High
☐	Assess if model architecture can be inferred via timing side-channels	*Side-channel analysis*	Medium
☐	Test if logit/probability outputs expose sufficient info for model cloning	*API testing*	High
☐	Check if API rate limits and cost controls deter bulk extraction attempts	*API testing*	High
	Model Artifact Security		
☐	Verify model weights are stored encrypted at rest	*Storage audit*	High
☐	Check access controls on model artifact repositories (S3, GCS, MLflow)	*Cloud IAM audit*	High
☐	Test if model serving endpoints expose raw weights or activations	*API testing*	Critical
☐	Verify model artifacts are signed and integrity-checked on load	*DevSecOps review*	High
	Inference API Protections		
☐	Test if inference API enforces strict authentication on all endpoints	*API testing*	High
☐	Check for unauthenticated or publicly exposed model endpoints	*Recon / scanning*	Critical
☐	Verify logging of all API calls to detect bulk extraction patterns	*Log review*	Medium
☐	Assess watermarking controls to detect stolen model outputs	*Output analysis*	Medium

Assessment Summary & Findings Tracker

Use this table to record confirmed findings during the engagement.

#	OWASP Category	Finding Title	Severity	Status	Remediation Owner
1				Open	
2				Open	
3				Open	
4				Open	
5				Open	

References & Resources

- OWASP Top 10 for LLM Applications: https://owasp.org/www-project-top-10-for-large-language-model-applications/
- MITRE ATLAS (AI Threat Landscape): https://atlas.mitre.org/
- NIST AI Risk Management Framework (AI RMF): https://www.nist.gov/artificial-intelligence
- Anthropic AI Safety Research: https://www.anthropic.com/research
- OWASP AI Exchange: https://owaspai.org/
- Google Secure AI Framework (SAIF): https://safety.google/cybersecurity-advancements/saif/

Responsible Use Disclaimer

This checklist is created for educational, research, and authorized security testing purposes only with an intent to help security professionals, developers, and organizations improve the security posture of systems that incorporate Large Language Models (LLMs) and AI technologies.
By accessing or using this document, you agree to the following:

- The information must only be used for lawful and ethical purposes.
- Security testing techniques described in these documents should only be performed on systems for which you have explicit authorization.
- The author does not endorse or support the use of this information for unauthorized access, exploitation, or malicious activities.
- Users are responsible for ensuring that their use of the materials complies with applicable laws, regulations, and organizational policies.
- The author provides these materials “as is” without warranties of any kind, and assumes no liability for any misuse, damages, or consequences arising from their use.

If you are unsure whether you are authorized to perform security testing on a system, do not proceed without obtaining proper permission from the system owner.